Bahasa Pemrograman Tingkat Rendah (MWCA010)

Program Studi Teknik Informatika ▪ Fakultas Teknologi Informasi ▪ Universitas Bale Bandung

Prosedur dan Manajemen Stack

Pertemuan 12

Dosen Pengampu

Cecep Suwanda, S.Si., M.Kom.

Semester Ganjil

Capaian dan Tujuan Pembelajaran

Sub-CPMK 3.4

Mengembangkan prosedur/subrutin dengan mekanisme pengiriman parameter (*parameter passing*) dan mengelola *stack frame* melalui instruksi `PUSH` dan `POP`, register `BP`, serta konvensi pemanggilan yang benar.

CPMK yang didukung: CPMK03 ▪ **CPL yang didukung: CPL03**

Setelah pertemuan ini, mahasiswa mampu:

1. menjelaskan alasan pemakaian prosedur beserta biaya pemanggilannya;
2. menjelaskan kerja `CALL` dan `RET` serta membedakan pemanggilan *near* dan *far*;
3. menelusuri isi stack sel per sel pada `PUSH`, `POP`, dan `CALL`;
4. membangun *stack frame* melalui *prologue* dan *epilogue* yang benar;
5. mengirim parameter melalui register, stack, dan variabel global beserta kelebihan serta kekurangannya;
6. menulis prosedur rekursif dan menjelaskan risiko *stack overflow*.

Peta Materi

1. **Prosedur, Pemanggilan, dan Pengembalian**
   alasan prosedur; biaya pemanggilan; `CALL` dan `RET`; alamat kembali `CS:IP`; pemanggilan *near* dan *far*

2. **Stack dan Operasi Stack**
   LIFO; `SS:SP`; arah tumbuh ke alamat rendah; `PUSH`, `POP`, `PUSHF`, `POPF`; penelusuran isi stack; `BP` penunjuk bingkai

3. **Stack Frame, Konvensi Pemanggilan, dan Rekursi**
   prologue dan *epilogue*; akses parameter; konvensi pemanggilan; nilai balik; rekursi dan *stack overflow*

Praktik pada pertemuan ini

Menulis prosedur berparameter dan memeriksa isi stack dengan *debugger* pada `prosedur-luas.asm` dan `faktorial.asm`.

BAGIAN

Prosedur, Pemanggilan, dan Pengembalian

Mengapa Prosedur Diperlukan

Program yang ditulis sebagai satu deretan instruksi panjang sulit dibaca, diuji, dan diperbaiki. Solusinya adalah **prosedur**.

1. **Modularitas.** Setiap prosedur menangani satu tugas yang batasnya jelas, sehingga program besar menjadi kumpulan bagian kecil yang dapat diuji terpisah.
2. **Penggunaan ulang** (*reuse*). Kode yang sering dipakai cukup ditulis sekali, misalnya prosedur pencetak bilangan heksadesimal.
3. **Abstraksi.** Pemanggil cukup mengetahui antarmuka prosedur: nama, parameter, dan nilai baliknya. Rincian di dalamnya dapat diganti tanpa mengubah pemanggil.

Prosedur dan `JMP`

`JMP` hanya memindahkan kendali tanpa menyediakan cara kembali; prosedur melengkapinya dengan mekanisme pengembalian.

Sumber: Bab XIII, Mengapa Prosedur Diperlukan

Definisi dan Biaya Pemanggilan Prosedur

Definisi

Prosedur (disebut juga **subrutin**) adalah sekelompok instruksi bernama yang dapat dipanggil dari bagian lain program, menerima parameter, dan dapat mengembalikan nilai kepada pemanggilnya.

Pemakaian prosedur tidak gratis. Setiap pemanggilan menimbulkan **biaya pemanggilan prosedur**:

Tahap	Siklus	Keterangan
`PUSH` parameter	11–15	setiap parameter menimbulkan akses memori
`CALL` prosedur	19	menyimpan alamat kembali lalu melompat
`RET`	16	mengambil alamat kembali dari stack

Untuk prosedur yang sangat sederhana, biaya ini dapat melebihi biaya pekerjaannya sendiri. Karena itu prosedur dipakai bukan untuk mengejar kecepatan, melainkan untuk kejelasan dan kemudahan pemeliharaan.

Sumber: Bab XIII, Mengapa Prosedur Diperlukan

Instruksi CALL dan RET

CALL mengerjakan dua hal sekaligus: menyimpan **alamat kembali** — nilai CS dan IP pada instruksi *sesudah* CALL — lalu memuat alamat prosedur ke CS dan IP.

```
CALL HitungLuas          ; simpan alamat kembali, lalu lompat
MOV  luas, AX            ; dieksekusi setelah RET
```

- Alamat kembali yang disimpan adalah alamat MOV luas, AX; RET memuatnya kembali ke IP.
- Stack dengan demikian merupakan bagian dari mekanisme kendali program, bukan sekadar tempat menyimpan data.

CALL tidak sama dengan JMP

JMP tidak menyimpan alamat kembali, sehingga RET akan mengambil alamat yang salah dari stack.

Pemanggilan *Near* dan *Far*

Aspek	`CALL` ***near***	`CALL` ***far***
Isi yang didorong ke stack	`IP` saja (2 byte)	`CS` lalu `IP` (4 byte)
Instruksi kembali	`RET` (`C3h`)	`RETF` (`CBh`)
Bentuk penulisan	`CALL HitungLuas`	`CALL LUAR:HitungLuas`
Pemakaian lazim	Prosedur pada satu berkas program	Prosedur pada pustaka atau *overlay* lain

Pada **pemanggilan dekat** (*near call*) prosedur berada pada segmen kode yang sama, sehingga hanya `IP` yang perlu disimpan. Pada **pemanggilan jauh** (*far call*) prosedur berada pada segmen lain, sehingga `CS` dan `IP` sama-sama disimpan. Seluruh contoh pada bab ini memakai pemanggilan *near*.

Sumber: Bab XIII, Pemanggilan Near dan Far

Rangkuman Bagian 1

- Prosedur dipakai untuk memperoleh modularitas, penggunaan ulang, dan abstraksi; `JMP` tidak dapat menggantikannya karena tidak menyediakan cara kembali.
- Biaya pemanggilan mencakup dorongan parameter (11–15 siklus per `PUSH`), `CALL` (19 siklus), dan `RET` (16 siklus) — sehingga prosedur dipakai demi kejelasan, bukan kecepatan.
- `CALL` menyimpan alamat kembali (`CS:IP` pada instruksi sesudah `CALL`) ke stack, lalu melompat; `RET` mengambil alamat itu kembali ke `IP`.
- Pemanggilan *near* menyimpan `IP` saja (2 byte) dan ditutup `RET`; pemanggilan *far* menyimpan `CS` dan `IP` (4 byte) dan ditutup `RETF`.

BAGIAN

Stack dan Operasi Stack

Stack: LIFO dan Arah Tumbuh

- **Stack** adalah daerah memori yang dipakai dengan aturan *last in, first out* (LIFO): data yang terakhir dimasukkan adalah data yang pertama dikeluarkan.
- Stack 8086 ditunjuk oleh pasangan `SS` dan `SP`; alamat fisik sel teratas adalah `SS` $\times$ `10h` + `SP`.
- **Penting:** Setiap penambahan isi stack menurunkan nilai `SP`, sehingga stack tumbuh menuju alamat yang semakin rendah.

Contoh angka

Bila `SS` = `1000h` dan ruang stack `100h`, maka `SP` mula-mula bernilai `FFFEh` — alamat tertinggi dari ruang yang disediakan direktif `.STACK` — dan alamat fisik sel teratasnya adalah `1FFFEh`.

Sumber: Bab XIII, Pengertian dan Arah Tumbuh Stack

Instruksi PUSH dan POP

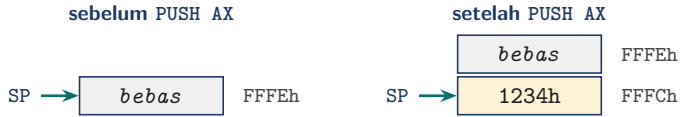
```
PUSH sumber     ; SP <- SP - 2, lalu isi sumber ke memori SS:SP
POP  tujuan     ; isi SS:SP ke tujuan, lalu SP <- SP + 2
```

- Keduanya selalu bekerja pada satu **kata** (*word*) berukuran 2 byte.
- Pada 8086, PUSH hanya menerima register atau isi memori 16-bit, sehingga PUSH 5 tidak sah.
- PUSHA dan POPA belum tersedia karena baru diperkenalkan pada prosesor 80186.

PUSHF dan POPF

PUSHF mendorong seluruh isi register FLAGS sebagai satu kata 16-bit, POPF memulihkannya. Dipakai bila prosedur mengubah *flag* untuk keperluan internal. Perlu diingat bahwa POPF ikut memulihkan IF, sehingga pemulihan yang salah dapat mematikan interupsi.

Jejak `PUSH AX` pada Stack



Keadaan awal SP = FFFEh dan AX = 1234h. Setelah PUSH AX, SP turun ke FFFCh; byte rendah 34h ditulis pada FFFCh dan byte tinggi 12h pada FFFDh — byte rendah selalu berada pada alamat yang lebih rendah.

Sumber: Bab XIII, Instruksi PUSH dan POP

Menelusuri Isi Stack pada Pemanggilan (1/2)

Keadaan awal: SS = 1000h, SP = FFFEh, stack kosong.

#	Instruksi	SP sesudah	Isi sel teratas	Keterangan
1	(keadaan awal)	FFFEh	kosong	SP menunjuk sel teratas
2	PUSH tinggi	FFFCh	000Ah	parameter kedua masuk lebih dahulu
3	PUSH lebar	FFFAh	0005h	parameter pertama masuk paling akhir
4	CALL HitungLuas	FFF8h	0102h	alamat kembali (IP) didorong

Urutan sel itu bukan kebetulan, melainkan akibat langsung dari urutan PUSH dan CALL: parameter yang didorong paling akhir berada paling dekat dengan BP.

Sumber: Bab XIII, Menelusuri Isi Stack pada Pemanggilan Prosedur

Menelusuri Isi Stack pada Pemanggilan (2/2)

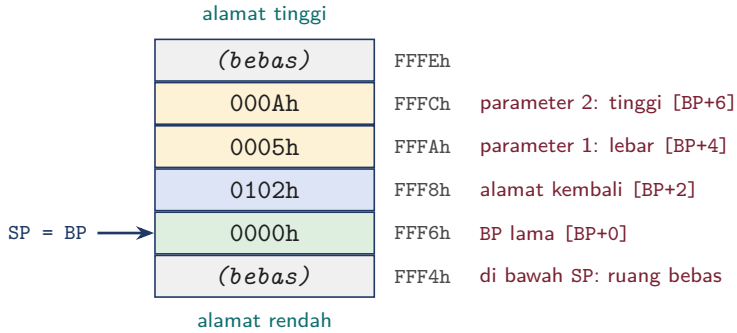
Lanjutan jejak: *prologue*, *epilogue*, dan `RET`.

#	Instruksi	SP sesudah	Isi sel teratas	Keterangan
5	`PUSH BP`	`FFF6h`	`0000h`	BP milik pemanggil disimpan
6	`MOV BP, SP`	`FFF6h`	`0000h`	BP = FFF6h; bingkai stack siap
7	`POP BP`	`FFF8h`	`0102h`	BP milik pemanggil dipulihkan
8	`RET 4`	`FFFEh`	kosong	IP diambil, lalu SP naik 4 byte

Langkah 1–4 terlihat pada frame sebelumnya. Perhatikan bahwa `MOV BP, SP` tidak mengubah SP: yang berubah hanyalah nilai BP sebagai titik acuan bingkai.

Sumber: Bab XIII, Menelusuri Isi Stack pada Pemanggilan Prosedur

Isi Stack Sel per Sel Setelah *Prologue*



Keadaan pada langkah keenam. Setiap sel berukuran 2 byte.

Sumber: Bab XIII, Menelusuri Isi Stack pada Pemanggilan Prosedur

Register BP sebagai Penunjuk Bingkai

- Akses parameter tidak dilakukan langsung melalui SP karena SP terus berubah setiap kali prosedur mendorong atau mengambil data.
- Register BP (*base pointer*) menyimpan salinan SP melalui `MOV BP, SP` dan tidak diubah oleh operasi PUSH maupun POP berikutnya.
- Karena itu jarak BP ke setiap parameter tetap konstan. Pola inilah yang disebut **bingkai stack** (*stack frame*).

Segmen bawaan BP adalah SS

`[BP+4]` selalu mengacu ke stack tanpa perlu awalan segmen. Cara ini merupakan penerapan mode pengalamatan *base-relative* yang dibahas pada Bab VIII.

Sumber: Bab XIII, Register BP sebagai Penunjuk Bingkai

Rangkuman Bagian 2

- Stack bekerja secara LIFO dan ditunjuk SS:SP; alamat fisik sel teratas = SS × 10h + SP.
- Stack 8086 tumbuh ke arah alamat rendah: PUSH menurunkan SP 2 byte lebih dahulu lalu menyimpan data, sedangkan POP mengambil data lebih dahulu lalu menaikkan SP.
- Setiap operasi bekerja pada satu kata (2 byte); byte rendah selalu berada pada alamat yang lebih rendah. PUSHF dan POPF menyimpan serta memulihkan register FLAGS.
- Penelusuran satu pemanggilan lengkap: dua PUSH parameter, CALL, PUSH BP, MOV BP, SP, lalu POP BP dan RET 4 mengembalikan SP ke FFFEh.
- BP dipakai sebagai titik acuan karena nilainya tidak berubah oleh PUSH/POP, sehingga jarak ke setiap parameter tetap.

BAGIAN

Stack Frame, Konvensi Pemanggilan, dan Rekursi

Prologue dan *Epilogue*

Prosedur selalu mengerjakan empat hal: membangun bingkai, mengakses parameter, menghasilkan nilai balik, lalu membongkar bingkai. Pola bakunya disebut **stack frame** standar.

```
HitungLuas PROC
    PUSH BP             ; prologue : simpan BP pemanggil
    MOV  BP, SP         ; prologue : BP <- titik acuan bingkai

    ; --- badan prosedur ---
    ; parameter diakses sebagai [BP+4], [BP+6], ...

    POP  BP             ; epilogue : pulihkan BP pemanggil
    RET                 ; epilogue : kembali ke pemanggil
HitungLuas ENDP
```

`PUSH BP` wajib ada karena `BP` hanya satu buah; tanpa menyimpannya, prosedur akan menimpa `BP` milik pemanggil. Pasangan `PUSH BP` dan `POP BP` harus seimbang pada setiap jalur keluar prosedur.

Sumber: Bab XIII, Prologue dan Epilogue

Mengakses Parameter dan Variabel Lokal

```
PUSH BP                 ; prologue
MOV  BP, SP
SUB  SP, 4              ; dua var lokal
MOV  WORD PTR [BP-2], 0
MOV  WORD PTR [BP-4], 0
MOV  SP, BP             ; epilogue
POP  BP
RET
```

Susunan bingkai

- [BP+0] — BP milik pemanggil
- [BP+2] — alamat kembali (IP)
- [BP+4] — parameter pertama
- [BP+6] — parameter kedua
- [BP+8] — parameter ketiga
- [BP-2] — variabel lokal pertama

Urutan PUSH menentukan nomor offset

Karena pemanggil mendorong parameter kedua lebih dahulu, parameter pertama-lah yang berada pada [BP+4]. Bila urutannya dibalik tanpa mengubah badan prosedur, kedua nilai akan tertukar tanpa pesan kesalahan apa pun.

Konvensi Pemanggilan: Tiga Cara Pengiriman Parameter

Cara	Kelebihan	Kekurangan	Contoh
Melalui register	tidak ada akses memori; paling cepat dan ringkas	jumlah parameter terbatas oleh jumlah register	`MOV AX, 5` lalu `CALL Tulis`
Melalui stack	jumlah parameter tidak terbatas; pemanggil tidak perlu tahu register yang dipakai prosedur	lebih lambat karena setiap parameter menimbulkan akses memori; rawan stack tidak seimbang	`PUSH 5` lalu `CALL HitungLuas`
Melalui variabel global	prosedur sangat sederhana; tidak ada instruksi tambahan	prosedur tidak dapat dipakai ulang; sulit ditelusuri saat kesalahan terjadi	`MOV lebar, 5` lalu `CALL HitungLuas`

Konvensi pemanggilan (*calling convention*) adalah kesepakatan tentang tiga hal: letak parameter, letak nilai balik, dan siapa yang membersihkan stack. Tanpa kesepakatan ini, prosedur yang ditulis satu orang tidak dapat dipakai oleh program orang lain.

Sumber: Bab XIII, Konvensi Pemanggilan

Siapa yang Membersihkan Stack

Prosedur membersihkan

Epilogue berakhir dengan `RET n`: mengambil `IP` lalu menambahkan `n` pada `SP` sekaligus.

```
POP  BP
RET  4          ; SP <- SP + 4
```

Pemanggil membersihkan

Prosedur memakai `RET` biasa, dan pemanggil menambahkan `ADD SP, 4` setelah `CALL`.

```
CALL HitungLuas
ADD  SP, 4      ; buang argumen
```

Jangan dipakai bersama-sama

`RET 4` yang disusul `ADD SP, 4` membuat `SP` naik 8 byte padahal hanya 4 byte yang perlu dibuang.

Sumber: Bab XIII, Konvensi Pemanggilan

Tanggung Jawab Penyimpanan Register

Golongan	Register	Tanggung jawab	Akibat bila dilanggar
Boleh dirusak (*caller-saved*)	`AX`, `BX`, `CX`, `DX`, `ES`	pemanggil menyimpan sendiri bila nilainya masih diperlukan	pemanggil memakai nilai yang sudah berubah
Wajib dipulihkan (*callee-saved*)	`BP`, `SI`, `DI`, `DS`, `SS`	prosedur menyimpan pada *prologue* dan memulihkan pada *epilogue*	pemanggil kehilangan nilai lama; akses data dapat salah alamat
Selalu dipulihkan	`SP`	stack harus seimbang saat `RET` dijalankan	`RET` mengambil alamat yang salah; program berhenti tak terkendali

Pada 8086 tidak ada aturan perangkat keras yang memaksa, sehingga kesepakatan dibuat di tingkat program; pembagian di atas mengikuti konvensi Microsoft C untuk 8086.

Sumber: Bab XIII, Konvensi Pemanggilan

Nilai Balik Prosedur

Nilai balik dikembalikan melalui register, bukan melalui stack.

Ukuran hasil	Register	Keterangan
8 bit	`AL`	hasil satu byte, misalnya pencarian karakter
16 bit	`AX`	ukuran lazim untuk prosedur 8086
32 bit	`DX:AX`	hasil `MUL` dan `IMUL` memang tersimpan di kedua register ini

Mengapa bukan melalui stack

`DX:AX` muncul secara alami karena `MUL` dan `IMUL` menyimpan hasil kalinya pada kedua register itu. `AX` termasuk register yang boleh dirusak, sehingga prosedur tidak wajib memulihkannya.

Sumber: Bab XIII, Nilai Balik Prosedur

Rekursi pada Assembly

```
Faktorial PROC
    PUSH BP               ; prologue
    MOV  BP, SP
    MOV  AX, [BP+4]       ; AX <- n
    CMP  AX, 1
    JBE  basis            ; kasus basis
    DEC  AX
    PUSH AX               ; dorong (n-1)
    CALL Faktorial        ; rekursi
    ADD  SP, 2            ; buang argumen
    MUL  WORD PTR [BP+4] ; AX <- (n-1)! * n
    POP  BP               ; epilogue
    RET
basis:
    MOV  AX, 1            ; 0! = 1! = 1
    POP  BP
    RET
Faktorial ENDP
```

- **Rekursi** adalah pemanggilan prosedur oleh dirinya sendiri.
- Pada bahasa tingkat tinggi rekursi tampak sebagai fitur bahasa; pada assembly rekursi tampak jelas sebagai penumpukan bingkai stack.
- Setiap pemanggilan menambah satu bingkai baru berisi parameter, alamat kembali, dan BP sendiri.
- Tanpa kasus basis, rekursi tidak akan berhenti dan stack akan melampaui batasnya.

Kedalaman Rekursi dan *Stack Overflow*

Pemakaian stack

- Setiap bingkai rekursif memerlukan 6 byte: 2 parameter, 2 alamat kembali, 2 `BP` lama.
- Untuk 5! terbentuk lima bingkai $\approx$ 30 byte, sehingga kebutuhan stack setara $O(n)$.
- Versi iteratif cukup satu bingkai saja ($O(1)$), tanpa biaya `CALL` dan *prologue* berulang.

Stack overflow

Prosesor 8086 tidak memiliki pengaman batas stack. Bila rekursi terlalu dalam atau kasus basis tidak pernah tercapai, `SP` terus turun melewati batas bawah ruang `.STACK`, lalu menimpa segmen data atau segmen kode — tanpa pesan kesalahan.

Fibonacci rekursif $F(n) = F(n-1) + F(n-2)$ jauh lebih berat: untuk $n = 20$ jumlah pemanggilannya melebihi `10.000`, sedangkan versi iteratifnya cukup satu perulangan. Karena itu Fibonacci jauh lebih baik ditulis secara iteratif pada assembly. Rekursi baru dipakai bila persoalannya memang bercabang, misalnya penelusuran pohon.

Sumber: Bab XIII, Rekursi pada Assembly

Kesalahan Umum Pengelolaan Stack

Empat kesalahan berikut paling sering dijumpai.

1. **Stack tidak seimbang.** Jumlah `PUSH` tidak sama dengan jumlah `POP` pada setiap jalur eksekusi, sehingga `RET` mengambil alamat yang salah.
2. **`RET` ke alamat yang salah.** Pemanggilan *near* ditutup dengan `RETF` atau sebaliknya, sehingga jumlah byte yang diambil dari stack tidak cocok.
3. **Lupa menyimpan register yang dipakai.** Prosedur memakai `SI` atau `DI` tanpa menyimpannya, sehingga pemanggil kehilangan nilai lama register itu.
4. **Kerusakan *stack frame*.** `BP` diubah di tengah prosedur, atau jalur keluar prosedur melompati `POP BP`, sehingga *epilogue* memulihkan nilai yang salah.

Sumber: Bab XIII, Kesalahan Umum Pengelolaan Stack

Rangkuman Bagian 3

- *Stack frame* standar dibangun *prologue* `PUSH BP` dan `MOV BP, SP`, lalu dibongkar *epilogue* `POP BP` dan `RET`.
- Parameter diakses melalui `[BP+4]`, `[BP+6]`, dan seterusnya; variabel lokal melalui `[BP-2]` setelah `SUB SP, n` pada *prologue*.
- Konvensi pemanggilan mengatur letak parameter, letak nilai balik, dan tanggung jawab pembersihan stack — melalui `RET n` oleh prosedur, atau `ADD SP, n` oleh pemanggil, tetapi tidak keduanya.
- Nilai balik diletakkan pada `AL`, `AX`, atau `DX:AX`. `AX`, `BX`, `CX`, `DX`, dan `ES` boleh dirusak; `BP`, `SI`, `DI`, `DS`, dan `SS` wajib dipulihkan.
- Rekursi pada assembly adalah penumpukan bingkai stack ($O(n)$) dan berisiko *stack overflow*; versi iteratif memakai stack tetap ($O(1)$).

Contoh 1: Prosedur `HitungLuas`

Menghitung luas persegi panjang dengan prosedur berparameter melalui stack; nilai balik pada `AX`, prosedur membersihkan sendiri argumennya dengan `RET 4`.

```
.MODEL SMALL
.STACK 100h
.DATA
    lebar  DW 5
    tinggi DW 10
    luas   DW 0
.CODE
START:
    MOV  AX, @DATA
    MOV  DS, AX
    PUSH tinggi         ; param kedua
    PUSH lebar          ; param pertama
    CALL HitungLuas     ; AX <- luas
    MOV  luas, AX
    MOV  AH, 4Ch
    INT  21h
```

```
HitungLuas PROC
    PUSH BP
    MOV  BP, SP
    MOV  AX, [BP+4]     ; lebar
    MUL  WORD PTR [BP+6]
    POP  BP
    RET  4              ; buang 4 byte
HitungLuas ENDP
END START
```

Hasil

$5 \times 10 = 50$ (0032h) pada `AX` dan variabel `luas`. Pemanggil *tidak* perlu `ADD SP, 4` karena `RET 4` sudah membuang kedua parameter.

Sumber: `prosedur-luas.asm` — prosedur ini hanya merusak `AX`.

Contoh 2: Faktorial Rekursif

Menghitung 5! secara rekursif; rekursi pada assembly adalah penumpukan bingkai stack.

```
.MODEL SMALL
.STACK 100h
.DATA
    n     DW 5
    hasil DW 0
.CODE
START:
    MOV  AX, @DATA
    MOV  DS, AX
    PUSH n              ; argumen n
    CALL Faktorial      ; AX <- n!
    ADD  SP, 2
    MOV  hasil, AX
    MOV  AH, 4Ch
    INT  21h
```

```
Faktorial PROC
    PUSH BP             ; prologue
    MOV  BP, SP
    MOV  AX, [BP+4]     ; AX <- n
    CMP  AX, 1          ; n <= 1 ?
    JBE  basis          ; kasus basis
    DEC  AX
    PUSH AX             ; dorong (n-1)
    CALL Faktorial      ; rekursi
    ADD  SP, 2          ; buang argumen
    MUL  WORD PTR [BP+4] ; AX <- (n-1)! * n
    POP  BP             ; epilogue
    RET
basis:
    MOV  AX, 1          ; 0! = 1! = 1
    POP  BP
    RET
Faktorial ENDP
```

Hasil: `AX` $= 120$ (`0078h`); lima bingkai $\times$ 6 byte $\approx$ 30 byte stack. Sumber: `faktorial.asm`.

Praktikum: Menulis Prosedur dan Memeriksa Stack

Judul praktikum

Menulis prosedur berparameter dan memeriksa isi stack dengan *debugger* ▪ **Sub-CPMK:** 3.4 ▪ **Waktu:** 100 menit ▪ **Bentuk:** individu ▪ **Tugas 6**

Alat dan bahan

- Emu8086 dengan jendela register, memori, dan stack, atau GUI Turbo Assembler
- DOSBox beserta `DEBUG`, TASM, dan TLINK
- Berkas `prosedur-luas.asm` dan `faktorial.asm`

Langkah kerja

1. Rakit dan jalankan `prosedur-luas.asm`; pastikan `AX` dan `luas` bernilai `0032h`.
2. Ubah `lebar` menjadi `7` dan `tinggi` `6`; pastikan hasilnya `42` (`002Ah`).
3. Telusuri isi stack per langkah; buktikan `[BP+4]` = `0005h` dan `[BP+6]` = `000Ah`.
4. Ubah `RET 4` menjadi `RET` biasa pada salinan berkas, lalu jelaskan akibatnya.

Dikumpulkan: `prosedur-luas.asm` berparameter tiga (memakai `[BP+8]`), `prosedur-ganjil.asm`, tabel pengamatan `SP`/`BP`/`SS`, tangkapan layar jendela stack, dan jawaban pertanyaan langkah 1(c), 3(b), 4(b).

Latihan

1. Sebutkan tiga alasan mengapa program sebaiknya dipecah menjadi prosedur. Jelaskan pula mengapa pemakaian prosedur tidak selalu membuat program lebih cepat, dan kapan prosedur sebaiknya dihindari.

2. Jelaskan apa yang dikerjakan `CALL` dan `RET`. Mengapa `JMP` tidak dapat dipakai sebagai pengganti `CALL`?

3. Diketahui `SS` = `2000h` dan `SP` = `FFFCh`. Tentukan alamat fisik sel teratas, lalu nilai `SP` serta isi sel setelah `MOV AX, 1234h`, `PUSH AX`, `PUSH BX`, dan `POP CX`. Berapa byte data yang masih tertinggal di stack?

4. Prosedur `Jumlah` memakai *prologue* `PUSH BP` dan `MOV BP, SP`, membaca `[BP+4]` dan `[BP+6]`, lalu ditutup `RET 4`. Tuliskan urutan instruksi pemanggil yang tepat, dan jelaskan mengapa pemanggil tidak boleh menambahkan `ADD SP, 4`.

5. Tulislah prosedur `HitungRataRata` yang menerima dua parameter 16-bit melalui stack dan mengembalikan hasil bagi keduanya (dibulatkan ke bawah) pada `AX`, lengkap dengan *prologue*, *epilogue*, dan program pemanggilnya.

Rangkuman

- Prosedur dipakai untuk memperoleh modularitas, penggunaan ulang, dan abstraksi. Biaya pemanggilannya berupa dorongan parameter, `CALL`, *prologue*, *epilogue*, dan `RET`.
- `CALL` menyimpan alamat kembali `CS:IP` ke stack lalu melompat; `RET` mengambil alamat itu dan mengembalikan kendali. Pemanggilan *near* menyimpan `IP` (2 byte) dan ditutup `RET`, pemanggilan *far* menyimpan `CS` dan `IP` (4 byte) dan ditutup `RETF`.
- Stack bekerja secara LIFO dan ditunjuk `SS:SP`, serta tumbuh ke arah alamat rendah: `PUSH` menurunkan `SP` 2 byte, `POP` menaikkannya kembali. `PUSHF` dan `POPF` menyimpan serta memulihkan `FLAGS`.
- *Stack frame* standar dibangun *prologue* `PUSH BP` dan `MOV BP, SP`, lalu dibongkar *epilogue* `POP BP` dan `RET`. Parameter diakses melalui `[BP+4]`, `[BP+6]`, dan seterusnya; variabel lokal melalui `[BP-2]`.
- Konvensi pemanggilan mengatur letak parameter, letak nilai balik, dan tanggung jawab pembersihan stack. Nilai balik diletakkan pada `AL`, `AX`, atau `DX:AX`; `AX`–`DX` dan `ES` boleh dirusak, sedangkan `BP`, `SI`, `DI`, `DS`, dan `SS` wajib dipulihkan.
- Rekursi pada assembly adalah penumpukan bingkai stack. Rekursi memakai stack $O(n)$ dan berisiko *stack overflow*, sedangkan versi iteratif biasanya memakai stack tetap $O(1)$.

Cek Pemahaman

1. Instruksi `CALL` *near* menyimpan ... ke dalam stack.
   1.1 register `CS` dan `IP`
   1.2 register `IP` saja
   1.3 seluruh register umum
   1.4 register `FLAGS` dan `BP`

2. Pada 8086, instruksi `PUSH` bekerja dengan cara ...
   2.1 menambah `SP` 2, lalu menyimpan data
   2.2 mengurangi `SP` 2, lalu menyimpan data
   2.3 menyimpan data, lalu menambah `SP` 2
   2.4 menyimpan data, lalu mengurangi `SP` 2

3. Setelah *prologue* dijalankan, parameter pertama yang didorong paling akhir oleh pemanggil berada pada ...
   3.1 `[BP+0]`
   3.2 `[BP+2]`
   3.3 `[BP+4]`
   3.4 `[BP+6]`

Penutup

Sebelum pertemuan berikutnya

- Selesaikan Tugas 6: `prosedur-luas.asm` berparameter tiga, `prosedur-ganjil.asm`, tabel pengamatan SP/BP/SS, dan tangkapan layar jendela stack.
- Ulangi bagian yang belum dikuasai dengan menelusuri `prosedur-luas.asm` dan `faktorial.asm` satu instruksi demi satu pada *debugger*.
- Baca **Bab XIV** — Interupsi BIOS/DOS dan Operasi I/O.

Bacaan lanjutan: Bab XIII bagian Konvensi Pemanggilan dan Rekursi pada Assembly; pembahasan mode pengalamatan *base-relative* pada Bab VIII; pembahasan segmen pada Bab VI.

Terima kasih

Bahasa Pemrograman Tingkat Rendah (MWCA010) ▪ Semester Ganjil